

Ishash

A corpus-hygiene utility for RAG data pipelines that identifies duplicate content risk, quantifies duplication with actionable statistics, and supports controlled remediation before indexing. It enables staged audit-then-cull workflows that improve retrieval quality, reduce embedding/indexing cost, and strengthen governance in knowledge curation operations.

Topic tags: rag, retrieval-augmented-generation, data-curation, data-governance, corpus-hygiene, document-deduplication, file-deduplication, knowledge-management, data-quality, bash, dotnet

Documentation map

- `README.md` : quick-start reference and command/flag summary
- `USERGUIDE.md` : step-by-step tutorial with practical workflows
- `ARCHITECTURE.md` : internal design and implementation architecture (Bash + .NET)

Features

- Sorts files alphabetically.
- Aligns the hash column based on the longest displayed file name.
- Supports multiple hash algorithms.
- Defaults to BLAKE3.
- Can recurse into subdirectories.
- Supports built-in exclusions plus user-defined exclusion patterns.
- Ignores `.dups/` directories by default.
- In recursive mode, processes and prints results directory-by-directory as traversal encounters them.
- Continues processing on per-file access errors and emits warnings instead of halting.
- Highlights adjacent matching hashes in green.
- Optional dedupe mode to keep one file and move duplicates into hidden `.dups/` directories.
- Prints a completion summary with duplicate counts and percentages.
- Supports macOS Catalina-compatible traversal behavior (no GNU `find -printf / sort -z` dependency).
- Built-in exclusions include common VCS/editor/temp artifacts and `*.lshash.json` sidecars.

Upfront use-case perspective

This tool was developed as a corpus-hygiene control for RAG pipelines.

In production RAG systems, duplicate files can create duplicate chunks, increase embedding/indexing spend, and over-weight repeated content during retrieval. That can reduce answer quality and make retrieval behavior less predictable.

The intended workflow is a staged curation process:

- Phase 1 (audit, no mutation): run without `-d` to profile duplication as part of pre-ingestion assessment. Use the completion statistics to quantify duplicate-file rate before chunking and embedding.
- Phase 2 (remediation, optional): run with `-d` (and optionally `--directory` for full-directory grouping) to quarantine duplicates into `.dups/`, reducing corpus redundancy before indexing.
- Phase 3 (post-curation validation): re-run audit and compare summary metrics to confirm that curation improved corpus quality.

This separation of discovery and action supports safer change control, clearer governance, and repeatable RAG data-preparation practice.

Script

- `lshash.sh`

Implementations

- Bash implementation:
 - Script: `lshash.sh`
 - Supports contiguous dedupe and `--directory` dedupe (with `--all-directory` as a compatibility alias)
- .NET implementation:
 - Project: `dotnet/`
 - Supports the same runtime options and dedupe variants as Bash

Requirements

- Bash 3.2+
- Standard Unix tools: `find`, `sort`, `awk`, `stat`, `mv`
- Hash command for selected algorithm:
 - `b3sum` for `blake3`
 - `sha256sum` for `sha256`
 - `sha512sum` for `sha512`
 - `sha1sum` for `sha1`
 - `md5sum` for `md5`
 - `b2sum` for `blake2`

macOS note

- The script now runs on macOS Catalina or later shell/tooling for traversal and sorting behavior.
- Hash command requirements still apply by algorithm choice. On macOS, `blake3` is typically the easiest path because `b3sum` can be auto-installed when package tooling is available.
- For non-BLAKE3 algorithms on macOS, the script prefers GNU `*sum` tools when installed, but automatically falls back to native commands where possible (`shasum` for `sha256 / sha512 / sha1` , and `md5` for `md5`).
- On legacy Bash (for example macOS system Bash 3.2), the script relaxes `nounset` (`set +u`) internally to avoid known empty-array expansion failures while preserving other strict-mode protections.

BLAKE3 auto-install behavior

If `blake3` is selected and `b3sum` is missing, the script attempts an automatic install using a detected package manager.

- Uses non-interactive elevation (`sudo -n`) when needed.
- Uses a timeout for install attempts.
- Timeout defaults to 20 seconds and can be overridden:

```
LSHASH_INSTALL_TIMEOUT=10 ./lshash.sh
```

If installation cannot be done automatically, the script exits with guidance.

.NET 10 implementation

This repository also includes a .NET 10 C# implementation with behavior parity to the Bash script.

Build a self-contained single-file executable

```
cd dotnet
./build.sh
```

Optional runtime identifier argument:

```
cd dotnet
./build.sh linux-x64
```

Output executable:

- `dotnet/dist/linux-x64/lshash`

The publish configuration is self-contained and single-file, so no .NET runtime is required on the target host. The .NET build also enables invariant globalization, so `libcicu` is not required on minimal Linux containers.

Build native macOS self-contained binaries

```
cd dotnet
./build-macos.sh
```

By default, `build-macos.sh` publishes `net6.0` binaries for better macOS Catalina compatibility.

Optional target selection:

```
cd dotnet
./build-macos.sh osx-arm64
./build-macos.sh osx-x64
./build-macos.sh --framework net10.0 osx-arm64
```

Output executables:

- `dotnet/dist/osx-arm64/lshash`
- `dotnet/dist/osx-x64/lshash`

macOS deployment for .NET implementation

If you prefer a containerized execution path, use the Docker deployment bundle:

```
cd dotnet/deploy/macos
./deploy.sh build
./deploy.sh audit /path/to/scan
./deploy.sh cull /path/to/scan
```

The deployment wrapper is documented in `dotnet/deploy/macos/README.md`.

Run from source

```
cd dotnet
dotnet run -c Release -- --help
```

.NET options

The .NET implementation supports the same options as Bash (`--algorithm` , `-r/--recursive` , `-e/--exclude` , `-d/--dedupe` , `--directory` (alias `--all-directory`), `--global` , `--prompt-delete` , `--move-dups` , `-q/--quiet` , optional `DIRECTORY`):

- `--directory` (alias: `--all-directory`)
 - With `-d/--dedupe` , dedupe by hash across all files in each directory, ignoring filename adjacency
 - Without `-d/--dedupe` , this flag is a no-op
- `--global`
 - With `-d/--dedupe` and `-r/--recursive` , dedupe by hash across the entire recursive tree
 - With `-d/--dedupe` without `-r/--recursive` , behaves like `--directory` on the selected directory
 - Sidecar metadata files `<moved-file>.lshash.json` are created only in recursive global mode (`-r -d --global`)
 - In dedupe mode, any directory containing `.lshash-exclude` is skipped with descendants
 - Without `-d/--dedupe` , this flag is a no-op
- `--prompt-delete`
 - With `-d/--dedupe` , after listing `.dups` directories, prompts `y/N` to delete them
 - Used alone (or with only `DIRECTORY`), recursively gathers existing `.dups` directories, lists them, and prompts `y/N` to delete them
 - When combined with other non-dedupe options, this flag is a no-op
- `--move-dups PATH / --move-dups=PATH`
 - Standalone mode (optionally scoped by `DIRECTORY`) that recursively finds existing `.dups` directories and moves files from them under `PATH` using original relative paths
 - Copying the resulting archive tree back onto the source tree restores duplicates (plus sidecars, when present)

.NET BLAKE3 backend selection

- Default backend is CPU.
- Override backend with environment variable `LSHASH_BLAKE3_BACKEND` :
 - `cpu` (default)
 - `gpu`
- If GPU backend initialization or hashing fails at runtime, the process falls back to CPU BLAKE3 for the remainder of that run.
- Optional GPU chunk budget override:
 - `LSHASH_BLAKE3_GPU_MAX_CHUNKS` (positive integer)
 - Default: `1048576` (`1 << 20`)

.NET performance tuning environment variables

- `LSHASH_DIAGNOSTICS=1` enables tuning diagnostics output.
- Network filesystems (for example `cifs`, `smb3`, `nfs`) auto-enable diagnostics even without `LSHASH_DIAGNOSTICS`.
- `LSHASH_HASH_WORKERS=<N>` pins a fixed worker count (disables adaptive worker tuning).
- `LSHASH_READ_BUFFER_KB=<N>` sets read buffer size for sequential hashing.

.NET examples

```
dotnet/dist/linux-x64/lshash -q
dotnet/dist/linux-x64/lshash -rq /path/to/scan
dotnet/dist/linux-x64/lshash -r -d shorter -q
dotnet/dist/linux-x64/lshash --directory           # no-op without -d
dotnet/dist/linux-x64/lshash -d shorter --directory
dotnet/dist/linux-x64/lshash -d shorter --global
dotnet/dist/linux-x64/lshash -r -d shorter --global
dotnet/dist/linux-x64/lshash -d shorter --prompt-delete
dotnet/dist/linux-x64/lshash --prompt-delete
dotnet/dist/linux-x64/lshash --prompt-delete /path/to/scan
dotnet/dist/linux-x64/lshash --move-dups /path/to/archive
dotnet/dist/linux-x64/lshash --move-dups=/path/to/archive /path/to/scan
```

Usage

```
./lshash.sh [--algorithm NAME] [-r|--recursive] [-e PATTERN] [--exclude PATTERN] [-d [MODE]] [--directory] [--global] [--prompt-delete] [--move-dups PATH] [-q|--quiet] [DIRECTORY]
```

macOS execution quick guide

Bash implementation (native, including Catalina)

```
cd /path/to/lshash
chmod +x ./lshash.sh
./lshash.sh --algorithm sha256 -r /path/to/scan
```

.NET implementation on modern macOS (native)

```
cd dotnet
./build-macos.sh
./dist/osx-arm64/lshash --help    # Apple Silicon
./dist/osx-x64/lshash --help     # Intel
```

.NET implementation on macOS Catalina (Docker Desktop)

```
cd dotnet/deploy/macos
./deploy.sh build
./deploy.sh audit /path/to/scan
./deploy.sh cull /path/to/scan
```

Options

- `--algorithm NAME`
 - Hash algorithm: `blake3`, `sha256`, `sha512`, `sha1`, `md5`, `blake2`
- `-r`, `--recursive`
 - Include files in subdirectories
 - Hidden `.dups/` directories are skipped by default
 - Output is emitted progressively per directory encountered during traversal
- `-e PATTERN`
- `--exclude PATTERN`
- `--exclude=PATTERN`
 - Exclude files matching glob pattern (repeatable)
 - Built-in exclusions are always active (for example `.dups` traversal skip, `.lshash-exclude`, `.git/.hg/.svn`, `.gitignore`, `.mdexplore-*.json`, `*.lshash.json`, and common temp/editor files)
- `-d [MODE]`, `--dedupe [MODE]`, `--dedup [MODE]`
- `-d=MODE`, `--dedupe=MODE`, `--dedup=MODE`
 - Dedupe files with identical hash in the same directory
 - Valid `MODE` values: `newer`, `older`, `shorter`, `longer`
 - Default mode when omitted: `shorter`
 - `shorter` / `longer` compare full root-relative path length (directory path + basename), not basename-only length
- `--directory` (alias: `--all-directory`)
 - With `-d/--dedupe`, uses full-directory hash grouping instead of contiguous-neighbor grouping
 - Without `-d/--dedupe`, no-op
- `--global`
 - With `-d/--dedupe` and `-r/--recursive`, dedupes by hash across all scanned files in the recursive tree (not per-directory)
 - With `-d/--dedupe` without `-r/--recursive`, behaves like `--directory` for the selected directory
 - In recursive global mode (`-r -d --global`), each moved duplicate gets a sidecar metadata JSON file `<moved-file>.lshash.json` in `.dups/` describing duplicate peers (full paths) and statuses (`kept` / `moved`)
 - In dedupe mode, any directory containing `.lshash-exclude` is skipped with descendants
 - Without `-d/--dedupe`, no-op
- `--prompt-delete`
 - With `-d/--dedupe`, after printing `.dups` directory paths, prompts `y/N` to delete them
 - Used alone (or with only `DIRECTORY`), recursively gathers existing `.dups` directories, lists them, and prompts `y/N` to delete them
 - When combined with other non-dedupe options, no-op
- `--move-dups PATH` / `--move-dups=PATH`

- Standalone mode (optionally scoped by `DIRECTORY`) that recursively finds existing `.dups` directories and moves files from them under `PATH` using original relative paths
- Copying that archive tree back over the source tree restores duplicates (plus sidecars, when present)
- `-q` , `--quiet`
 - Only print duplicate lines (the lines that would be highlighted green in normal output)
 - Works with and without dedupe, and with and without recursive mode
- `DIRECTORY` (optional positional argument)
 - Scan this directory instead of the current working directory
 - Output paths remain relative to the selected directory root
- One-letter short switches are stackable in any order (for example `-rd` , `-dr` , `-rq` , `-re` `'*.log'`).

Output formatting

- Hash values are left-justified in a single aligned column.
- If the previous listed file has the same hash, the current hash is shown in green.
- When dedupe moves a file, the file name is italicized and annotated:
 - `(moved to .dups/)`
- Completion summary reports duplicate count and percentage of scanned files.
- With `-r/--recursive` , summary also reports directories traversed.
- With `-d/--dedupe` , summary wording changes to duplicates “found and moved”.

Dedupe behavior

When dedupe is enabled:

- Primary use case: remove copy/restore/merge artifacts where duplicate files usually sort next to each other (for example names containing `(copy)` , version suffixes, or sync-conflict tags).
- Duplicate groups are determined by contiguous same-hash blocks in alphabetical listing order within each directory.
- Files that cannot be hashed are skipped for block matching, so they do not break a contiguous duplicate block among hashable neighbors.
- Genuine executable program files are excluded from dedupe matching and never moved (requires execute permission plus program/script detection, for example MIME types such as `application/x-pie-executable` or `text/x-shellscript` ; shebang scripts are also treated as executable programs even if MIME resolves to `text/plain` ; many file managers show these as `Program`).
- One file is kept in place based on selected mode.
- All other duplicates in that directory are moved to that directory's `.dups/` subdirectory.
- In recursive mode, dedupe is still per directory encountered during traversal.
- Tie-breaking rule: first file in sorted listing order is kept.

- If a destination name already exists in `.dups/` , a `.dupN` suffix is added.
- `--directory` provides a more thorough filename-blind mode that checks duplicates across the full directory. It only takes effect when used with `-d/--dedupe` .
- `--global` extends dedupe scope across the full recursive tree when combined with `-d` and `-r` , and writes provenance JSON sidecars (`<moved-file>.lshash.json`) for moved files.

Dedupe scope matrix

Flags	Duplicate scope	Grouping method	Moved file destination	Sidecar metadata
<code>-d</code>	Per directory	Contiguous same-hash runs in sorted filename order	Same directory <code>.dups/</code>	No
<code>-d --directory</code>	Per directory	Full-directory hash grouping (filename adjacency ignored)	Same directory <code>.dups/</code>	No
<code>-d --global</code>	Selected directory only	Full-directory hash grouping (same as <code>--directory</code>)	Same directory <code>.dups/</code>	No
<code>-d -r --global</code>	Full recursive tree	Whole-tree hash grouping across directories	Each file's own source directory <code>.dups/</code>	Yes (<code>.lshash.json</code>)

Global mode metadata (`<moved-file>.lshash.json`)

In recursive `--global` mode (`-r -d --global`), every moved duplicate gets a sidecar metadata file next to it in `.dups/` :

- Name: `<moved-file>.lshash.json`
- Location: same `.dups/` directory as the moved file
- Purpose: explain the duplicate set peers and which file was kept vs moved

JSON structure:

```
{
  "hash": "<hash>",
  "dedupeMode": "shorter",
  "subject": {
    "path": "/abs/path/to/dir/.dups/file.ext",
    "status": "moved"
  },
  "others": [
    {
      "path": "/abs/path/to/kept/file.ext",
      "status": "kept"
    },
    {
      "path": "/abs/path/to/another/dir/.dups/file2.ext",
      "status": "moved"
    }
  ]
}
```

Dedupe flow diagrams

Technical flow diagrams are maintained in `ARCHITECTURE.md`.

Strategy summary

- Default (`-d`): optimized for copy/restore/merge artifacts where duplicate names are often alphabetically adjacent.
- `--directory` with `-d` : more thorough and filename-blind dedupe across the entire directory.
- `--directory` without `-d` : no-op (normal non-dedupe listing behavior).
- `--global` with `-d -r` : cross-directory, whole-tree hash dedupe with per-moved-file metadata JSON.
- `--global` with `-d` (no `-r`): equivalent dedupe scope to `--directory` on the selected directory and does not emit sidecar JSON.

Examples

Basic listing (default BLAKE3)

```
./lshash.sh
```

Use SHA-256

```
./lshash.sh --algorithm sha256
```

Recursive listing

```
./lshash.sh -r
```

Exclude multiple patterns

```
./lshash.sh -r -e '*.log' --exclude '*.tmp' --exclude='build/*'
```

Dedupe with default mode (**shorter**)

```
./lshash.sh -d
```

Dedupe and keep newest file

```
./lshash.sh -r --dedupe newer
```

Dedupe and keep longest full relative path

```
./lshash.sh --dedupe=longer
```

Global dedupe in one directory (non-recursive)

```
./lshash.sh -d shorter --global /path/to/scan
```

This uses full-directory hash grouping for that single directory (same scope behavior as **--directory**) and does not write sidecar metadata.

Global dedupe across full recursive tree

```
./lshash.sh -r -d shorter --global /path/to/scan
```

This compares hashable files across all directories in the tree, moves losers to each file's local `.dups/`, and writes `<moved-file>.lshash.json` sidecars.

Global dedupe with a different keep policy

```
./lshash.sh -r --dedupe newer --global /path/to/scan
```

In each duplicate set, the newest file is kept in place and all others are moved to their source-directory `.dups/` folders.

Inspect generated sidecar metadata

```
find /path/to/scan -path '*/*.dups/*.lshash.json' -maxdepth 6 -print  
cat /path/to/scan/some/dir/.dups/example.txt.lshash.json
```

If `jq` is available:

```
jq . /path/to/scan/some/dir/.dups/example.txt.lshash.json
```

Only show duplicate lines

```
./lshash.sh -q  
./lshash.sh -rq /path/to/scan
```

Prompt-delete garbage collection mode

```
./lshash.sh --prompt-delete  
./lshash.sh --prompt-delete /path/to/scan
```

Rehydrate duplicates from existing `.dups` into an archive tree

```
./lshash.sh --move-dups /path/to/archive  
./lshash.sh --move-dups=/path/to/archive /path/to/scan
```

Summary message examples (hypothetical)

These examples use made-up file sets to show how the completion summary text changes by mode.

1. Audit pass (no `-d`): duplicates found

Hypothetical files in one directory:

```
a.txt      (content: same)
b.txt      (content: same)
c.txt      (content: different)
```

Command:

```
./lshash.sh --algorithm sha256
```

Expected output shape:

```
a.txt <hash-A>
b.txt <hash-A>
c.txt <hash-C>
Summary: scanned 3 file(s); 1 duplicate file(s) were found (33.33% of scanned files).
```

2. Recursive audit (`-r` , no `-d`): adds traversed directories

Hypothetical tree:

```
./a.txt      (content: same)
./b.txt      (content: same)
./sub/c.txt  (content: unique)
```

Command:

```
./lshash.sh --algorithm sha256 -r
```

Expected output shape:

```
a.txt      <hash-A>
b.txt      <hash-A>
sub/c.txt  <hash-C>
Summary: scanned 3 file(s); 1 duplicate file(s) were found (33.33% of scanned files); 2 directories were traversed.
```

3. Cull pass (-d): duplicates found and moved

Hypothetical files in one directory:

```
a.txt      (content: same)
aa.txt     (content: same)
aaa.txt    (content: same)
```

Command:

```
./lshash.sh --algorithm sha256 -d shorter
```

Expected output shape:

```
a.txt      <hash-A>
aa.txt (moved to .dups/)  <hash-A>
aaa.txt (moved to .dups/) <hash-A>
Summary: scanned 3 file(s); 2 duplicate file(s) were found and moved (66.66% of
scanned files).
```

Expected result on disk:

```
.dups/aa.txt
.dups/aaa.txt
```

4. Audit pass with no duplicates: zero percentage

Hypothetical files in one directory:

```
a.txt      (content: alpha)
b.txt      (content: bravo)
c.txt      (content: charlie)
```

Command:

```
./lshash.sh --algorithm sha256
```

Expected output shape:

```
a.txt <hash-A>
b.txt <hash-B>
c.txt <hash-C>
Summary: scanned 3 file(s); 0 duplicate file(s) were found (0.00% of scanned
files).
```

5. Recursive cull (**-r -d**): moved count plus traversed directories

Hypothetical tree:

```
./a.txt          (content: same)
./aa.txt         (content: same)
./sub/p.txt      (content: same)
./sub/pp.txt     (content: same)
```

Command:

```
./lshash.sh --algorithm sha256 -r -d shorter
```

Expected output shape:

```
a.txt          <hash-A>
aa.txt (moved to .dups/) <hash-A>
sub/p.txt      <hash-P>
sub/pp.txt (moved to .dups/) <hash-P>
Summary: scanned 4 file(s); 2 duplicate file(s) were found and moved (50.00% of
scanned files); 2 directories were traversed.
```

6. **--directory** without **-d** : modifier no-op

Hypothetical files in one directory (non-adjacent duplicate content):

```
a-copy.txt      (content: same)
m-middle.txt    (content: unique)
z-sync.txt      (content: same)
```

Command:

```
./lshash.sh --algorithm sha256 --directory
```


Expected output shape:

```
a-copy.txt <hash-S>
m-middle.txt <hash-M>
z-sync.txt <hash-S>
Summary: scanned 3 file(s); 0 duplicate file(s) were found (0.00% of scanned
files).
```

7. `--directory` with `-d` : non-adjacent duplicates moved

Use the same hypothetical files as example 6.

Command:

```
./lshash.sh --algorithm sha256 -d shorter --directory
```

Expected output shape:

```
a-copy.txt <hash-S>
m-middle.txt <hash-M>
z-sync.txt (moved to .dups/) <hash-S>
Summary: scanned 3 file(s); 1 duplicate file(s) were found and moved (33.33% of
scanned files).
```

8. Quiet mode (`-q`) still prints summary

Hypothetical files in one directory:

```
a.txt (content: same)
b.txt (content: same)
c.txt (content: unique)
```

Command:

```
./lshash.sh --algorithm sha256 -q
```

Expected output shape:

```
b.txt <hash-A>
```

```
Summary: scanned 3 file(s); 1 duplicate file(s) were found (33.33% of scanned files).
```

Notes

- Dedupe moves files; it does not delete them.
- Review output carefully before running dedupe on important directories.

Troubleshooting

Default run seems slow or pauses

- First run with `blake3` may try to auto-install `b3sum` if missing.
- Use another algorithm immediately:

```
./lshash.sh --algorithm sha256
```

- Reduce install wait time:

```
LSHASH_INSTALL_TIMEOUT=5 ./lshash.sh
```

`b3sum` not found

- Install it manually, or use another algorithm.
- Example fallback:

```
./lshash.sh --algorithm sha512
```

Permission or file access errors

- If a file cannot be read (for hash or metadata), the tool prints a warning and continues.
- Output for those files shows `<hash unavailable>`.
- In dedupe mode, inaccessible files are ignored for contiguous block matching; hashable neighbors can still form a duplicate block across them.

Permission issues during auto-install

- Auto-install uses non-interactive sudo (`sudo -n`) and will fail fast if credentials are not already available.
- Fix by installing `b3sum` manually or run with a different algorithm.

Dedupe did not move files as expected

- Dedupe only groups contiguous same-hash neighbors (in alphabetical listing order) within the same directory.
- With `-r` , grouping is still per directory, not across the entire tree.
- For cross-directory dedupe across the full tree, use `-r -d --global` .
- Confirm mode selection:
 - `newer` keeps newest
 - `older` keeps oldest
 - `shorter` keeps shortest full root-relative path (default)
 - `longer` keeps longest full root-relative path

Quiet mode printed nothing

- `-q/--quiet` only prints duplicate lines (green lines in normal mode).
- If no adjacent duplicate hashes are encountered in listing order, quiet output will be empty.

Unexpected shell warnings about current directory

- If your shell says it cannot access the current directory (`getcwd` warnings), your working directory may have been deleted.
- Change into a valid directory before running again:

```
cd /home/npepin/Projects/lshash
```

macOS error: cannot make pipe for process substitution: Too many open files

- Recent versions of the script avoid high-frequency process substitutions in recursive traversal/sorting paths to prevent file-descriptor exhaustion on legacy macOS Bash.
- If you still see this, ensure you are running the latest script revision from this repository.

FAQ

How do I run a simple hash listing in the current directory?

```
./lshash.sh
```

How do I scan a different directory?

```
./lshash.sh /path/to/scan  
./lshash.sh -rq /path/to/scan
```

How do I recurse but skip common noise directories and file types?

```
./lshash.sh -r -e '.git/*' -e '.dups/*' -e 'node_modules/*' -e '*.log' -e  
'*.tmp'
```

How do I use a non-BLAKE3 algorithm quickly?

```
./lshash.sh --algorithm sha256
```

How do I dedupe recursively and keep the newest file in each duplicate set?

```
./lshash.sh -r --dedupe newer
```

How do I dedupe across the entire recursive tree (not per-directory)?

```
./lshash.sh -r -d shorter --global /path/to/scan
```

For each moved duplicate in recursive global mode (`-r -d --global`), a sidecar file `<moved-file>.lshash.json` is created in `.dups/` with peer paths and `kept / moved` status.

How do I dedupe but keep the shortest full relative path instead?

```
./lshash.sh -d
```

This uses `shorter`, which compares full root-relative path length.

How do I archive existing `.dups` content without re-running dedupe?

```
./lshash.sh --move-dups /path/to/archive  
./lshash.sh --move-dups=/path/to/archive /path/to/scan
```

`--move-dups` writes files under the archive using original relative paths, so copying that archive tree back over the source tree restores duplicates (plus sidecars, when present).

Where do moved duplicates go?

- Duplicates are moved into a hidden `.dups/` subdirectory under the same directory where the duplicate was found.

What if I want dedupe aliases?

- All of these are accepted:
 - `--dedupe`
 - `--dedup`
 - `-d`

Regression tests

Run the parity/regression checks (Bash + .NET):

```
chmod +x tests/regression.sh  
./tests/regression.sh
```

For hash-algorithm rationale and comparison notes, see the BLAKE3 appendix in `ARCHITECTURE.md`.